Week 05 Rust Minimax Search Tree Analysis

Project Research Design and Implementation of a Gomoku Game Engine with Artificial Intelligence
Academic module Basic Project
Report language English
Evidence review date 17 October 2026

1 Research purpose and evidence boundary

This week studies how deterministic Minimax search turns a board evaluation into a decision against an opponent who chooses the least favourable continuation. The central research problem is to distinguish an error in recursive search from a change in the game tree supplied to it. Searching every legal move and searching only nearby candidates are different experiments, even when both use the same Minimax function.

The work reconstructs the argument from inspected Rust source and reproducible local evidence. It replaces earlier timing tables and named horizon fixtures that lacked traceable execution records. Earlier figures such as a claimed 18 candidates or a universal depth three timeout are not retained as measured results.

Evidence labels distinguish **[A]** inspected source or recorded execution, **[B]** primary literature, **[C]** deduction or mathematical example, and **[D]** proposed work. The reference crate is inspectable; its search modules are not thereby integrated into the browser game. The game checkout has HEAD `c1e7447532e4c5ff5590f2c5562dd21cd0acf7b9`; unrelated App, language, and Wink changes remain outside this report.

2 Research questions and conceptual background

Three questions guide the investigation.

1. Does reference Minimax compute the specified finite depth value while preserving its input state?
2. How does replacing all legal moves with a declared candidate policy alter the tree and computational cost?
3. What can a terminal result establish that a heuristic result at the search horizon cannot establish?

[B] Shannon distinguishes exhaustive continuation to the end of a game from practical lookahead that evaluates intermediate positions. His mathematical discussion maximizes a move after minimizing the opponent response, and warns that static evaluation can mislead during an unfinished combination. These ideas motivate separate treatment of terminal utility, depth limits, and tactical instability here. They provide no performance measurements for this project. Shannon 1950 Programming a Computer for Playing Chess.

3 Source model and responsibility

[A] The implementation is `caro-core-reference/src/search/minimax.rs`, specifically `search_minimax`, `value`, and `terminal_score`. Configuration comes from `SearchConfig` and `MoveMode` in `src/search/mod.rs`. These are the actual identifiers; the earlier report described a `SearchSpaceMode` and `MinimaxConfig` API that is not the current contract.

File and symbol	Responsibility and observed behaviour
`src/state.rs` `GameState::play`	Rejects finished occupied or invalid moves and derives win or draw

File and symbol	Responsibility and observed behaviour
`src/state.rs` `GameState::undo`	Restores cell side status last move and count from an undo token
`src/search/mod.rs` `generate_moves`	Selects all legal or candidates without changing the recurrence
`src/movegen.rs` `generate_candidates`	Emits empty nearby cells in row major order then optionally truncates
`src/evaluator.rs` `evaluate`	Combines contiguous pattern scores from a fixed perspective
`src/metrics.rs` `SearchStats`	Counts traversal leaf evaluation generation and ordering separately

Paths in this table are relative to `caro-core-reference`. The state owner is `GameState`; search owns temporary traversal, and the evaluator reads state. The lifecycle is generate moves, play, inspect terminal or recurse, undo, and compare. A terminal move can leave the stored side unchanged, so terminal detection must precede assumptions about the next player.

4 Mathematical definition of the searched value

[C] Let $s=(B,t,R,z)$ contain board, side to move, rule profile, and terminal status. Let p be the fixed root player, h remaining depth, and ℓ moves from the root. Let $T(s,a)$ be a legal successor and $G(s)$ the configured move set. Every recursive value remains in the perspective of p.

[A] The implemented terminal score uses $M=10\,000\,000$:

$$U_p(s,\ell)=\begin{cases} M-\ell, & z=Won(p),\\ -M+\ell, & z=Won(opponent(p)),\\ 0, & z=Draw. \end{cases}$$

[C] Let $H_p(s)$ be configured static evaluation. Write $E(s,h)$ for the condition that the state is nonterminal, $h>0$, and $G(s)$ is nonempty. The recurrence is:

$$V_{p,h}(s,\ell)=\begin{cases} U_p(s,\ell), & s \text{ terminal},\\ H_p(s), & s \text{ nonterminal},\ h=0,\\ H_p(s), & s \text{ nonterminal},\ h>0,\ G(s)=\varnothing,\\ max_{a\in G(s)}V_{p,h-1}(T(s,a),\ell+1), & E(s,h),\ t=p,\\ min_{a\in G(s)}V_{p,h-1}(T(s,a),\ell+1), & E(s,h),\ t\neq p. \end{cases}$$

The empty generated set fallback matters: radius zero or a zero candidate cap can leave no candidates on a nonempty playable board. That condition is not itself a draw. In all legal mode, a properly maintained full board is terminal before generation.

[A] The root retains the first move in its deterministic list when scores tie, because it replaces an existing best move only for strict score improvement. Optional ordering changes that list, so deterministic does not mean independent of configuration.

5 Correctness argument and state restoration

[C] The recurrence terminates because depth decreases on every edge and terminal states stop earlier. Correctness relative to the supplied tree follows by induction. At depth zero or a terminal node, the

implementation returns the specified leaf value. Assume every child returns the specified value at depth $h-1$. The maximizing parent compares all generated legal children and takes their maximum; the minimizing parent takes their minimum. Its result therefore equals the recurrence at depth h.

This proves finite tree evaluation, not a forced win in unrestricted Gomoku. It assumes generated moves are legal, transitions follow the chosen rules, and undo restores the parent before visiting another sibling. Omitting a move can invalidate a claim about the full game without invalidating this induction about the restricted tree.

[A] `UndoToken` stores the previous player, status, last move, count, and played coordinate. Clearing only the cell would be insufficient: a sibling could inherit a terminal status or wrong player. `tests/search_equivalence.rs::search_does_not_mutate_root_state` checks board, side, status, count, and last move after Alpha Beta. Execution evidence must identify the actual algorithms checked; a test for one is not automatically proof for every search path.

The pseudocode summarizes current Minimax. The parent side determines whether combine means maximum or minimum.

```
VALUE state h root ply
  count this node
  if terminal return terminal utility
  if h equals zero return heuristic
  moves = generate configured moves
  if moves empty return heuristic
  optionally order moves
  best = minus infinity for MAX otherwise plus infinity
  for move in moves
    token = play move; child = VALUE state h minus one root ply plus one
    undo token; best = combine best child
  return best
```

6 Worked decision and horizon example

[C] Consider a two ply tree with heuristic leaves from the root perspective. Move A lets the opponent choose 8 or 2; move B allows 5 or 4:

$$V(A)=min(8,2)=2, V(B)=min(5,4)=4, V(s)=max(2,4)=4.$$

The root chooses B. Selecting the largest leaf, 8, would ignore the opponent choice. The example establishes the order of quantifiers: the root chooses, and the opponent responds after observing that choice.

A separate constructed horizon example has immediate evaluations $H(A)=80$ and $H(B)=30$. Extending one ply gives A response values -100 and 60, and B values 20 and 25. Depth one chooses A; depth two compares -100 with 20 and chooses B. These are synthetic values, not engine outputs or measured Gomoku fixtures. They show why depth can change a decision without establishing a particular failure in the application.

[A] A terminal win reached at the horizon is still scored as terminal because `terminal_score` precedes depth zero evaluation. Conversely, a favourable nonterminal heuristic is not a certificate that every later opponent response loses.

7 All legal and candidate search spaces

[C] On a nonterminal board with n occupied cells and no forbidden moves,

$$A(s)=\{a : B(a)=0\}, |A(s)|=225-n.$$

For nonempty boards, the uncapped radius policy is

$$C_r(s)=\{a \in A(s) : \exists x \text{ occupied}, \ \|a-x\|_\infty \le r\}.$$

[A] The empty board is special: the generator returns only centre `(7,7)`. Otherwise it emits marked cells in row major order and applies `max_candidates` before search ordering. The default is radius two without a cap. A single central stone has eight radius one neighbours and twenty four radius two neighbours. These follow from the neighbourhood geometry; they are not an average branching factor.

[C] Candidate mode restricts both players. Removing a root option can hurt the root; removing an opponent refutation can help it. Candidate value is therefore neither a general upper nor lower bound on all legal value. Radius is a hypothesis about relevance. A row major cap can remove an immediate tactical move even when it lies within the radius.

For a uniform illustrative branching factor b, unpruned search contains $1 + b + \cdots + b^h$ nodes. With four stones and no early terminal branch, all legal depth two contains $1 + 221 + 221 \cdot 220 = 48\,842$ nodes. This conditional count is not a runtime measurement; terminal branches can reduce it. Real branching varies by position and ply.

8 Experiment method and reproducibility

[D] Comparisons must record rule profile, fixture stones and side, depth, mode, radius, cap, weights, ordering, executable build mode, and toolchain. To isolate move policy, keep algorithm and evaluator fixed. To isolate Alpha Beta next week, keep move policy fixed instead. These controls answer different questions.

The shared package is `research/week_05_09`. Run `python3 research/week_05_09/run.py` from the workspace root. It runs locked native tests and the release harness; exact commands, source hashes, clock scope, and toolchain are retained in provenance. The run used Rust 1.98.1 on Linux x86 64 and an Intel Xeon W 11855M. Search timing excludes state cloning and table or key initialization. Controlled fixtures have three repetitions; sixteen seeded reachable positions have one. Fixture construction is in `research/week_05_09/src/main.rs::fixture`. Nodes include the root. `candidate_moves_generated` sums move list lengths at expanded nodes; it is not a unique board count. `ordering_evaluations` is separate work and must remain visible when discussing cost.

Recorded timing describes the local native harness configuration. It does not establish browser responsiveness, Worker cancellation, device latency, or a universal depth threshold. Timing speedup requires comparable elapsed measurements; a node ratio measures traversal reduction.

9 Recorded local results

[A] The summary CSV records the following Minimax results for the clustered fixture: X at `(7,7)` and `(8,7)`, O at `(7,8)` and `(8,8)`, with X to move. Coordinates are zero based. Ordering is disabled and the rule is Freestyle.

Move mode	Depth	Nodes	Leaves	Score	Move
All legal	2	48842	48620	0	6 7
Radius 1 no cap	2	189	176	0	6 7
Radius 1 cap 12	2	157	144	0	6 7
Radius 1 cap 12	3	1885	1728	4910	6 8

The depth two all legal count matches the conditional calculation in Section 7. Candidate mode gives the same score and move for this fixture at depth two, but that agreement is local. The depth three row illustrates a changed horizon and must not be used as a controlled speed comparison with depth two.

[A] A separate cap loss record demonstrates omission: on the win fixture, radius one without a cap yields score 9999999 at depth one. Cap six removes both winning endpoints `(7,2)` and `(7,7)` and

yields 99990. Thus fewer nodes do not imply preserved tactical decisions. This is a move policy limitation, not a Minimax recurrence defect.

[A] The verification record reports 112 same configuration Minimax and Alpha Beta score and move comparisons and 551 root restoration checks across the harness, including budget abort probes. Comparison counts include repeated controlled runs. Nine existing native crate tests passed in the test log. No named historical horizon fixture is claimed to have been executed.

10 Interpretation and limitations

[A] Inspection establishes finite depth Minimax traversal, explicit move mode selection, terminal priority, tie retention, and paired play and undo. Existing integration tests include `alpha_beta_matches_minimax_value_and_move`, `search_does_not_mutate_root_state`, and `immediate_win_is_selected`. Their tactical fixture is manually constructed; it does not represent a tournament or complete legal game history.

[C] One scoring proof obligation remains. `evaluate` clamps nonterminal scores at $M - 1$, while search scores a win at ply ℓ as $M - \ell$. For $\ell > 1$, a hypothetical heuristic near that clamp can outrank a later win. The clamp comment therefore does not establish absolute terminal dominance after distance adjustment. This is source derived, not a reproduced application failure. A safe nonterminal limit would be below $M - L$ for maximum permitted root ply L.

[C] Pattern evaluation counts maximal contiguous runs and open ends. It does not fully model broken threats such as `XX.XX`. Greater depth can expose some missing tactical consequences, but finite depth and a small corpus cannot establish complete threat handling. Empty candidate fallback, candidate caps, and fixture legality require separate checks.

11 Conclusions and next research step

Minimax evaluates the declared finite tree when transitions, leaf scoring, and restoration satisfy their contracts. All legal mode supplies a small depth reference for separating traversal correctness from move omission. Candidate mode supplies a smaller tree whose tactical coverage needs independent measurement.

[D] The next step compares Alpha Beta on this same tree with identical leaf rules, ordering, and tie policy, then investigates candidate recall and deeper tactical fixtures. Browser integration requires an explicit adapter and runtime verification. This report does not establish deployment, training, playing strength, or device behaviour.